



Published under the Creative Commons Attribution-ShareAlike 4.0 license.



Literate Programming: Дональд Кнут, WEB и современные рабочие процессы

Sam Starling

Zhovten Games, sam@phantom-draft.com

 0009-0009-2621-6372

Oksana Dubinetska

Zhovten Games, oksana.dubinetska@phantom-draft.com

 0009-0003-8777-8412

Abstract: В статье Literate Programming 1984 года Дональд Кнут предложил писать программы прежде всего для человеческого чтения и лишь затем для машинного исполнения. Настоящий обзор реконструирует эту аргументацию через систему WEB и прослеживает её актуальность для генерации исходного кода, исполняемых документов, воспроизводимых окружений и LLM-assisted разработки.

Keywords: литературное программирование, Дональд Кнут, WEB, программная инженерия, воспроизводимые исследования, исполняемые документы, LLM-assisted разработка, Prompt-Literate Workflow, literate design

Методологическая рамка обзора

Широкий спектр затрагиваемых вопросов не случаен. Статья Дональда Кнута — не узкая документация к системе WEB, а методологический текст практикующего программиста, который выступает одновременно как инженер и идеолог профессии. Обсуждая конкретный инструмент, Кнут затрагивает устройство программы, порядок её объяснения, стиль, переносимость, стоимость разработки, публикацию кода и будущую форму программных сред. Поэтому современный комментарий к этой работе неизбежно выходит за пределы реконструкции WEB и требует сопоставления нескольких инженерных линий.

При этом перечисленные в обзоре инструменты и практики не рассматриваются как единое генеалогическое древо, где каждый современный механизм буквально происходит от WEB. Для дальнейшего анализа важно различать три типа связей. Первый тип — прямое или исторически близкое продолжение WEB: прежде всего CWEB и noweb. Второй тип — инструментально родственная практика: например, Org Babel tangling, где используются weaving/tangling-механики, но это само по себе не доказывает прямую генеалогическую связь. Третий тип — смысловая ассоциация, или продуктивное сопоставление: WEB ↔ notebooks, WEB ↔ workflows на основе больших языковых моделей (LLM)¹, WEB ↔ navigation в интегрированной среде разработки (IDE)² и context maps. В последнем случае речь идёт о повторном возникновении сходной задачи: связать код, объяснение, порядок чтения, навигацию и проверяемый результат.

Такое сопоставление соответствует логике, которую сам Дональд Кнут описывает в разделе Related Work: сила WEB состояла не в изобретении каждого элемента по отдельности, а в соединении уже существовавших идей в целостный рабочий метод.

¹ Large language model, или LLM, — статистическая модель, обученная продолжать и преобразовывать текст по контексту и инструкциям.

² Integrated development environment, или IDE, — среда, объединяющая редактирование, навигацию, запуск, отладку и другие инструменты разработки.

Оглавление

Основная статья

- **Вступление** — историческая постановка проблемы, рамка сравнения и обоснование современного remaster-подхода.
- **A. Introduction** — смена адресата программирования: от инструктажа машины к объяснению человеку.
- **B. The WEB System** — WEB как система, порождающая человеческую и машинную проекции из одного источника.
- **C-F. Example / source / tangle / weave** — пример с простыми числами, исходный WEB-файл, машинная проекция TANGLE и человеческая проекция WEAVE.
- **G. Additional Bells and Whistles** — дополнительные возможности WEB и переход от учебного примера к производственной сложности.
- **H. Occam's Razor** — ограничение инструмента и необходимость методологической сдержанности.
- **I. Portability** — переносимость literate-подхода за пределы исходной пары TeX / Pascal.
- **J. Programs as Webs** — программа как сеть именованных фрагментов, зависимостей и объяснительных связей.
- **K. Stylistic Issues** — стиль, именование секций и читаемость literate source.
- **L. Economic Issues** — стоимость внедрения, сопровождения и дисциплины literate programming.
- **M. Related Work** — родственные линии: structured programming, документационные системы и executable-document практики.
- **N. Retrospect and Prospects** — ретроспективная оценка и перспективы метода.
- **Выводы 1-5** — что в идее Кнута сохраняется буквально, что смещается в reproducible research и notebook-среды, а что заново проявляется в LLM-assisted workflow.

Навигация по репозиторию

Канонический источник статьи

- публичный корень репозитория: <https://github.com/Zhovten-Games/literate-programming>;
- canonical source обзора: 01-LiterateProgramming, WEB / LiterateProgramming, WEB .md;

- файл авторских метаданных: `01-LiterateProgramming`, WEB
B `/authors.yml`;
- инвентарный слой библиографии: `01-LiterateProgramming`, WEB
B `/Bibliography/Original.md`;
- sectioned bibliography JSON: `01-LiterateProgramming`, WEB
B `/Bibliography/*.json`.

Companion-набор

- корень companion-набора: `02-Literate-Companion-Implementations/`;
- общий README companion-набора: `02-Literate-Companion-Implementations/README.md`;
- Makefile для проверочных запусков: `02-Literate-Companion-Implementations/Makefile`;
- английская ветка примеров: `02-Literate-Companion-Implementations/examples/en/`;
- русская ветка примеров: `02-Literate-Companion-Implementations/examples/ru/`.

Детерминированные companion-примеры 01-06

- `01-cweb` — более канонический CWEB-маршрут с явной двухветочной моделью `tangle/weave`;
- `02-noweb-like` — основной облегчённый C++ remaster, ориентированный на `tangle-first workflow`;
- `03-org-babel` — plain-text literate workflow в Org-mode / Emacs-среде с tangling;
- `04-quarto` — executable-document маршрут для публикационного HTML / computational publishing;
- `05-jupyter` — notebook-маршрут с отдельным вниманием к выполнению, скрытому состоянию и reproducible execution;
- `06-rmarkdown` — reproducible report workflow на R Markdown / knitr / Pandoc.

Prompt-Literate Workflow и LLM-границный пример

- публичный репозиторий методологии Prompt-Literate Workflow: <https://github.com/IRONCREED/prompt-literate-workflow>;
- роль: переиспользуемый репозиторий методологии и GitHub template repository;
- интеграция в literate-programming: подключённый submodule/scaffold в корне репозитория `prompt-literate-workflow/`;
- путь методологии, используемый companion-примерами: `02-Literate-Companion-Implementations/methodology/prompt-literate-workflow/`;
- локальная специализация статьи для задачи Кнута о простых числах: `02-Literate-Companion-Implementations/methodology/extensions/prim-es-example/`;
- `07-prompt-literate` — демонстрационное применение Prompt-Literate Workflow (PLW) к примеру с простыми числами, а не определение самого метода.

Рабочие файлы внутри 07-prompt-literate

- `COMPANION.md` — входная точка companion-примера;
- `primes.plan.md` — human-authored plan и canonical source для prompt-literate примера;
- `CONTRACTS.md` — контракты чанков для prime-number task;
- `SCENARIOS.md` — сценарии проверки и критерии принятия;
- `prompts/fill-chunks.prompt.md` — bounded prompt для заполнения только разрешённых LLM-TODO чанков;
- `prompts/review-generated-code.prompt.md` — prompt для review, а не для переписывания архитектуры;
- `generated/.gitkeep` — пустой каталог для будущего принятого generated artifact;
- `tests/smoke-check.sh` — проверка принятого generated-кода;
- `output.expected.txt` — ожидаемые маркеры вывода;
- `TRACE.md` — журнал модели, prompt-run, review, ручных правок, проверки и решения о принятии.

Основная статья

Вступление

Историческое значение статьи Дональда Кнута *Literate Programming* особенно отчётливо проявляется в ретроспективе. Текст был опубликован в 1984 году — до позднейшей канонизации паттерн-языка в *Design Patterns* (1994) (Gamma и др. 1994) и задолго до *Clean Architecture* (2017) (Martin 2017). В этом смысле перед нами одна из ранних попыток методологически упорядочить сложность программной разработки не только через структуру кода, но и через структуру его объяснения.

Статью *Literate Programming* можно рассматривать как точную постановку проблемы, сохраняющей актуальность: программа должна быть не только исполнимой, но и объяснимой. Дональд Кнут предлагает изменить адресата программирования: вместо того чтобы считать главной задачей инструктаж компьютера, программист должен прежде всего объяснять человеку, что он хочет заставить компьютер сделать (Knuth 1984). Это смещение не отменяет машинного исполнения, но подчиняет его более широкой задаче — организации программы в порядке человеческого понимания.

В 2026 году подобная постановка приобретает дополнительную актуальность, поскольку изменилась сама рабочая связка разработки. Если классическая модель может быть описана как движение от человека к человеку и затем к машине, то в AI-assisted workflow всё чаще возникает цепочка, где человек формулирует намерение для машины, а одна машинная система помогает подготовить материал для другой. Однако prompt сам по себе не является literate source. Современный AI-вариант можно считать содержательным продолжением literate programming только при сохранении инженерной дисциплины: объяснение → спецификация → код → тесты → артефакт. Без этой связки ускоряется не столько понимание программы, сколько производство кода с недостаточной проверяемостью и слабой объяснительной структурой.

В данном обзоре WEB рассматривается как исходная форма этой идеи. У Дональда Кнута один источник порождает две проекции: документ для человека и программу для машины. Именно эта связка — объяснение, код и механизм сборки между ними — служит главным критерием дальнейшего разбора. При этом задача обзора не состоит в том, чтобы воспроизвести Pascal-пример как самоценный исторический объект. Ремастер кодовой базы примера выполнен на C++, а основной демонстрационный маршрут выбран в noweb-like форме: он сохраняет named chunks и tangling, но не требует входа в более тяжёлый CWEB/TeX-контур.

Дальнейший разбор сопоставляет несколько инструментальных и методологических линий:

- CWEB (Knuth и Levy, б. д.),
- noweb-like C++ (Ramsey, б. д.),
- Org Babel («Org Manual: Extracting Source Code», б. д.),
- Quarto / Jupyter / R Markdown («Quarto Documentation», б. д.; «Project Jupyter Documentation», б. д.; «R Markdown Documentation», б. д.).

Технические развилки вынесены в companion-документы внутри репозитория с кодовой базой; основной текст следует за статьёй Дональда Кнута и проверяет её центральную идею: можно ли организовать программу в порядке человеческого понимания, а уже затем получить машинно-исполняемый код.

В финале обзор возвращается к нескольким вопросам: как соотнести literate programming с последующим языком инженерных паттернов; какие утверждения Кнута сбылись буквально; какие идеи сместились в область reproducible research и notebook-сред; и что сегодня заново возникает вокруг LLM. Отдельно будет показано, что LLM должна получать не свободный запрос, а ограниченную операцию над human-authored literate plan, chunk contracts, bounded prompts, tests/smoke-check и TRACE.

Полноценного companion 07-llm, равного CWEB, noweb, Org Babel, Quarto, Jupyter или R Markdown по воспроизводимости, здесь нет: LLM-генерация зависит от модели, версии, контекста и режима выполнения. Вместо этого добавлен граничный 07-prompt-literate, демонстрирующий применение Prompt-Literate Workflow. Его статус и ограничения будут подробно рассмотрены в выводе.

Prompt-Literate Workflow был первоначально сформулирован в рамках настоящего обзора, а затем уточнён при практическом применении к независимому ядру игрового проекта. Этот пилот показал необходимость разделить компактное универсальное ядро метода и локальный для проекта слой расширений. Доменные правила не должны молча переопределять базовые инварианты: они оформляются как локальные расширения, а включение локального правила в основу требует отдельного прохода по его обобщению и продвижению в ядро методологии.

A. Introduction

Дональд Кнут начинает с постановки профессиональной нормы. Структурное программирование уже сделало программы надёжнее и понятнее, но этого недостаточно. Следующий шаг он видит не в новой управляющей конструкции, а в изменении способа изложения программы.

Образ программиста-эссеиста здесь не декоративен. Понимание должно быть встроено в устройство программы: понятия вводятся в порядке, удобном читателю, а формальные и неформальные средства дополняют друг друга.

B. The WEB System

Во втором разделе Дональд Кнут показывает, что *literate programming* у него не метафора, а конкретная система: WEB. Её назначение — соединить язык документа и язык программирования так, чтобы один source мог порождать два разных результата: читаемое описание для человека и исполняемую программу для машины.

Дональд Кнут предлагает осмыслить сложную программу как сеть простых частей и связей между ними.

Здесь полезно развести три термина:

- **WEB** — от англ. *web* (сеть, паутина). У Дональда Кнута это программа как сеть именованных секций и отношений между ними, а не просто линейный файл.
- **TANGLE** — от глагола *to tangle* (спутывать, переплестать в машинный порядок). Утилита собирает компиляторно-ориентированный исходник программы.
- **WEAVE** — от глагола *to weave* (ткать, сплестать в читаемый документ). Утилита строит человеко-ориентированный документ о программе.

Технически WEB двуязычен и в нём заложена переносимость идеи за пределы конкретного стека. В исходной версии это TeX как язык форматирования и Pascal как язык программирования, но Дональд Кнут прямо подчёркивает, что принцип не привязан к этой паре: вместо TeX могли бы быть Scribe или Troff, вместо Pascal — C, LISP, FORTRAN, ALGOL, ассемблер и другие языки.

Ключевая схема раздела — **WEAVE / TANGLE**. WEAVE создаёт документ, который описывает программу и облегчает сопровождение; TANGLE создаёт машинно-исполняемую программу. Оба результата генерируются из одного источника, поэтому код и документация не расходятся как две независимые правды и в ручной синхронизации не нуждаются.

Эта схема и задаёт критерий для современных ветвей, обозначенных во вступлении: нас интересуют не сами инструменты, а то, сохраняют ли они связь между объяснением, кодом и проверяемым артефактом.

C–F. Пример с простыми числами: source, tangle и weave

В этом блоке принят ранее заявленный методологический выбор: основной remaster выполняется в noweb-like C++. Такой формат выбран не из-за ограничений CWEB, а как облегчённая адаптация идеи Дональда Кнута к современному командному workflow. Даже базовая дисциплина canonical source -> tangling -> generated artifact -> smoke-check уже имеет ненулевой входной порог; обязательная отдельная ветка генерации читаемого документа делает пример ближе к классическому WEB/CWEB, но хуже подходит для демонстрации минимального практического ядра.

Более канонический контраст сохранён в 02-Literate-Companion-Implementations/examples/ru/01-cweb/: это прямая C-семейная линия от WEB к CWEB (Knuth и Levy, б. д.) с двухветочной моделью primes.w -> ctangle -> primes.c -> cc/gcc -> primes -> output.txt и primes.w -> cweave -> primes.tex -> pdftex -> primes.pdf. Тем самым показано, что полная tangle/weave-модель остаётся работоспособной; однако для основного обзора выбран 02-noweb-like, чтобы показать переносимость принципа без обязательного входа в CWEB/TeX-контур.

Исторически пример Дональда Кнута стоит после дискуссии о structured programming: вопрос уже не только в том, как структурировать поток управления, а в том, как структурировать объяснение программы (Dijkstra 1972, pp. 26–39).

Поэтому подпункты C–F ниже не повторяют весь код. Они фиксируют четыре проекции примера у Дональда Кнута и сразу сопоставляют их с нашим remaster-пайплайном. Полный noweb-like source приведён один раз — в подразделе «Ремастер».

С. Читаемый пример у Дональда Кнута

В разделе С представлена читаемая проекция примера, которая затем также обсуждается в разделе F: читатель получает не линейный Pascal-файл, а композицию программы как объяснение.

```

1000      :
1.      ;
2.      ;
3.      ;
4.      ;
5.      ,      .

```

D. Исходный WEB-файл

Раздел D показывает исходный PRIMES.WEB, то есть материал, который автор пишет напрямую: документационный слой + Pascal + команды WEB.

В нашем remaster-эквиваленте роль источника истины выполняет `primes.nw`.

Маркеры и синтаксис чанков noweb:

- `<<program>>=` определяет именованный чанк с именем `program`.
- `<<name>>` ссылается на другой именованный чанк.
- `@` завершает текущий чанк.
- `notangle -Rprogram` выбирает `program` как корневой чанк.
- `notangle` разворачивает ссылки на чанки и генерирует обычный C++.
- C++ компилятор никогда не видит noweb-маркеры; он видит только сгенерированный `primes.cpp`.

E. Машинная проекция: TANGLE

Раздел E фиксирует машинную проекцию: TANGLE преобразует PRIMES.WEB в PRIMES.PAS.

В нашем remaster аналогом является `primes.cpp`.

```
primes.nw -> notangle -Rprogram -> primes.cpp -> g++ -> primes ->
output.txt
```

`output.txt` выступает как проверочный артефакт (smoke-check), а не как читаемый документ.

F. Человеческая проекция: WEAVE

Раздел F задаёт отдельную ветку читаемого документа: в WEB/CWEB это WEAVE/cweave -> TeX -> PDF.

Ремастер: почему noweb-like версия намеренно облегчена

02-noweb-like представляет tangle-first C++ remaster. Он сохраняет минимальное практическое ядро метода: canonical literate source, named chunks, generated machine artifact и проверочный артефакт smoke-check. Отдельная ветка генерации читаемого документа в этом варианте не является обязательной.

Ниже собрана единая literate-форма remaster-материала: объяснение остаётся рядом с соответствующими чанками, а не вынесено в отдельные повторяющиеся разделы.

Верхний уровень задаёт намерение программы и порядок композиции:

```
# Printing primes: a C++ reboot

This program prints the first 1000 prime numbers.
The top-level structure is deliberately simple:
generate the prime table, then print it.

<<program>>=
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <vector>

<<constants>>
<<types>>
<<declarations>>

int main() {
    const auto primes = generate_primes(PRIME_COUNT);
    print_table(primes);
    return 0;
}
```

```
<<prime-generation>>
<<table-output>>
@
```

Параметры вывода фиксируют layout таблицы и сохраняют связь с оригинальным примером Дональда Кнута:

```
The output format follows Knuth's original example:
50 rows per page, 4 columns, 10 character positions per column.

<<constants>>=
constexpr std::size_t PRIME_COUNT = 1000;
constexpr std::size_t ROWS_PER_PAGE = 50;
constexpr std::size_t COLUMNS_PER_PAGE = 4;
constexpr int COLUMN_WIDTH = 10;
@
```

Контейнер для таблицы простых чисел оформлен как отдельная смысловая роль:

```
The table of primes is represented as a growing sequence.
The alias makes the role of the container explicit.

<<types>>=
using PrimeTable = std::vector<int>;
@
```

Объявления функций вводят разделение ответственности между генерацией, проверкой кандидата и выводом:

```
Generation, candidate checking, and output are introduced
as separate operations.

<<declarations>>=
PrimeTable generate_primes(std::size_t count);
bool is_prime_candidate(int candidate, const PrimeTable& primes);
void print_table(const PrimeTable& primes);
@
```

Генерация простых чисел сохраняет алгоритм remaster-версии без изменения логики:

```
<<prime-generation>>=
PrimeTable generate_primes(std::size_t count) {
    PrimeTable primes;
    if (count == 0) return primes;
    primes.push_back(2);

    for (int candidate = 3; primes.size() < count; candidate +=
2) {
        if (is_prime_candidate(candidate, primes)) {
            primes.push_back(candidate);
        }
    }
    return primes;
}

bool is_prime_candidate(int candidate, const PrimeTable& primes)
{
    for (int p : primes) {
        if (p > candidate / p) return true;
        if (candidate % p == 0) return false;
    }
    return true;
}
@
```

Фаза вывода сохраняет постраничную раскладку и вычисление индексов по строкам/колонкам:

```
<<table-output>>=
void print_table(const PrimeTable& primes) {
    std::size_t page_number = 1;
    std::size_t page_offset = 0;

    while (page_offset < primes.size()) {
        std::cout << "The First " << primes.size()
```

```

        << " Prime Numbers --- Page " << page_number <<
        "\n\n";

        for (std::size_t row = 0; row < ROWS_PER_PAGE; ++row) {
            for (std::size_t column = 0; column <
COLUMNS_PER_PAGE; ++column) {
                const std::size_t index = page_offset + row +
column * ROWS_PER_PAGE;
                if (index < primes.size()) {
                    std::cout << std::setw(COLUMN_WIDTH) <<
primes[index];
                }
            }
            std::cout << '\n';
        }

        std::cout << '\n';
        ++page_number;
        page_offset += ROWS_PER_PAGE * COLUMNS_PER_PAGE;
    }
}
@

```

`primes.nw` остаётся canonical literate source; `primes.cpp` выступает как generated machine artifact; `output.txt` — как smoke-check output.

G. Additional Bells and Whistles

В разделе G Дональд Кнут делает переход: пример с простыми числами хорош для демонстрации, но не доказывает пригодность WEB для реальной разработки. Система должна выдерживать не только учебный фрагмент, но и большие программы, разные компиляторы, нестандартные соглашения, типографику, переносимость и накопление технических исключений. Иначе это не метод, а красивая игрушка для статьи.

Здесь Дональд Кнут перечисляет возможности WEB83, которые не понадобились в PRIMES.WEB, но нужны в крупных программах: ручное управление форматированием WEAVE, пропуск фрагментов через TANGLE почти «как есть», управление пробелами и началом строки, восьмеричные и шестнадцатеричные константы, кодирование

символов через ASCII, строковый пул с контрольной суммой, числовые макросы и простая арифметика времени трансляции.

Смысл этого раздела не в самих фичах. Большая часть из них сегодня выглядит как компенсация ограничений Pascal и старых компиляторов. В C++ часть проблем просто исчезает: есть нормальные строки, `constexpr`, локальные переменные, стандартные контейнеры, форматтеры и сборочные системы.

Смысл в том, что `literate`-инструмент должен уметь работать с грязной реальностью языка и платформы, а не только с идеальным примером. Пример с простыми числами объясняет принцип, а не весь `production`-контур.

Для `noweb`-like C++-ремастера это можно сформулировать так:

```
      :
<<program>>
<<constants>>
<<types>>
<<prime-generation>>
<<table-output>>

      :
<<headers>>
<<platform-specific-code>>
<<configuration>>
<<generated-bindings>>
<<tests>>
<<diagnostics>>
<<build-notes>>
```

То есть «дополнительные украшения» — не украшения в обычном смысле. Это слой, который позволяет `literate source` оставаться главным источником даже тогда, когда проект перестаёт быть чистым и маленьким. Где такая дисциплина действительно оправдана, будет разобрано в выводе нашего обзора; сам Дональд Кнут дальше тоже ограничивает универсальность WEB и не описывает его как инструмент для всех случаев.

H. Occam's Razor

Раздел H выполняет функцию методологического ограничения по отношению к предыдущему разделу. После длинного списка возможностей легко сделать ложный вывод, будто literate-система должна непрерывно расти и встраивать в себя всё: форматирование, переносимость, макрорасширения, публикацию, сборку, диагностику, окружение. Дональд Кнут, напротив, напоминает принцип бритвы Оккама: сложность должна появляться только там, где она действительно необходима для задачи, а не потому, что инструмент технически «умеет ещё».

Это особенно важно для судьбы WEB, сила которого изначально заключалась в том, что он связывал объяснение и код программы в единый рабочий контур. Но та же система исторически обросла ритуалами и входным порогом: множество команд, типографический слой, зависимость от привычек конкретной школы разработки. Поэтому в поздних ветвях возникла другая реакция — не расширять ядро бесконечно, а сокращать его до минимально жизнеспособной дисциплины.

В этом смысле `noweb`, используемый в нашем ремастере, важен как инженерная редукция, а не как «анти-WEB». Он сохраняет ключевую связку именованных фрагментов и `tangling`, но убирает значительную часть церемониальности. Это не отменяет классический WEB и не делает `noweb` универсальным решением; это показывает, что literate-подходу нужна мера. Если система пытается охватить все задачи одновременно, она начинает проигрывать собственной сложности раньше, чем начинает системно улучшать понимание.

I. Portability

В разделе I Дональд Кнут переходит от удобства изложения к переносимости. TeX и связанное с ним программное обеспечение должны были работать на множестве машин и операционных систем. WEB оказался полезен именно потому, что позволял сохранять общий источник программы и при этом учитывать различия платформ.

Сначала Дональд Кнут описывает `bootstrapping`-схему: имея `TANGLE.PAS`, `TANGLE.WEB` и `WEAVE.WEB`, можно получить рабочий `TANGLE`, затем с его помощью получить `WEAVE.PAS`, а после этого уже строить всю WEB-систему и программы вроде TeX.

Далее Дональд Кнут уточняет практические ограничения модели. Реальные машины различаются наборами символов, файловыми соглашениями, терминалами, упаковкой данных, арифметикой с плавающей точкой и качеством компиляторов. Поэтому ожидать, что один `TANGLE.PAS` или один `TANGLE.WEB` будет без изменений

работать везде, методологически некорректно. Системно-зависимые изменения неизбежны.

Решение WEB — **change files**. TANGLE и WEAVE читают не только основной WEB-файл, но и отдельный .CH-файл с заменами для конкретной системы. В результате master source остаётся стабильным, а платформенные правки живут отдельно. Дональд Кнут подчёркивает, что TANGLE.WEB фактически не меняется: изменяется TANGLE.CH, а базовая логика TANGLE остаётся общей для всех.

Подход Дональда Кнута к literate programming не оторван от сборки и распространения.

Для нашего noweb-like C++-ремастера это можно передать так:

```
canonical source:
  primes.nw

generated artifact:
  primes.cpp

platform overlay:
  primes.linux.patch
  primes.windows.patch
  config/platform.hpp
  CMake presets
```

Иными словами, прямой аналог .CH сегодня не обязан быть отдельным «change file» в классическом формате WEB. В современной практике круг продуктивных аналогий шире:

```
change file → patch layer
change file → platform adapter
change file → environment-specific config
change file → build preset
change file → overlay
```

Но принцип тот же: не ломать основной объяснительный источник ради каждой платформенной особенности. Если для Windows, Linux или конкретного компилятора нужны различия, они должны быть вынесены в слой адаптации. Иначе canonical source быстро превращается в набор платформенных исключений, где основная идея теряется за адаптационным шумом.

Современные ветви реализуют этот принцип разными средствами: от master-source дисциплины до управления воспроизводимой средой.

Различие между классическим WEB и executable-document ветвью здесь становится предметным. У Дональда Кнута переносимость — это прежде всего перенос программы между машинами. В Quarto/Jupyter/R Markdown переносимость часто означает другое: возможность пересобрать вычислительный отчёт с теми же зависимостями, теми же данными и тем же порядком выполнения. Это не буквальный .CN, но та же тревога: источник должен переживать смену среды.

В современной разработке к этому добавляется ещё один слой: переносимость уже не ограничивается различиями между операционными системами и компиляторами. Всё чаще речь идёт о воспроизводимой рабочей среде: WSL³, контейнерах⁴, cloud IDE⁵, CI/CD runners⁶, dev containers⁷ и управляемых окружениях. В такой ситуации вопрос меняется с «соберётся ли программа на другой машине» на «можно ли восстановить весь контур: source, tooling, зависимости, команды сборки, проверочные артефакты и порядок выполнения».

Показательно, что companion-примеры этого обзора были воспроизведены внутри WSL-среды под Windows 11. Это современная форма portability: Windows выступает рабочей платформой, WSL — Linux-исполнительной средой, а сами примеры проверяются через конкретные CLI-инструменты⁸ — ctangle, notangle, emacs, quarto, jupyter, Rscript, pandoc, g++. Тем самым переносимость становится не абстрактным свойством исходного текста, а проверяемым состоянием всей toolchain-связки.

Хорошо видно, как изменилась роль окружения. В классическом WEB системно-зависимые различия выносились в .CN-файлы; в современной практике аналогичную функцию могут выполнять WSL-профиль, контейнер, lockfile⁹, CI-конфигурация,

³ Windows Subsystem for Linux, или WSL, — функция Windows, позволяющая запускать Linux-окружение и Linux-инструменты без отдельной настройки виртуальной машины или dual boot.

⁴ Контейнер — изолированная среда выполнения, где приложение запускается вместе с явно заданными зависимостями и настройками.

⁵ Cloud IDE — среда разработки, работающая в браузере или на удалённой инфраструктуре, а не только на локальной машине автора.

⁶ CI/CD runner — исполнитель автоматизированного пайплайна проверки, сборки или публикации проекта.

⁷ Dev container — контейнер, описывающий воспроизводимое окружение именно для разработки: инструменты, расширения, зависимости и команды проекта.

⁸ Command-line interface, или CLI, — управление инструментом через команды терминала, что удобно для воспроизводимых проверок и автоматизации.

⁹ Lockfile фиксирует точные версии зависимостей и связанные метаданные, чтобы повторная установка не зависела от случайных обновлений.

build preset¹⁰ или документация воспроизводимого запуска. Принцип остаётся тем же: основной объяснительный источник не должен распадаться из-за особенностей конкретной машины. Теперь рядом с source нужно фиксировать не только платформенную правку, но и способ восстановить саму среду исполнения.

J. Programs as Webs

Раздел J возвращает к центральной архитектурной идее статьи: программа устроена не как простая линейка шагов и не как аккуратное дерево «сверху вниз». Дональд Кнут предлагает представлять её как сеть — web — где именованные фрагменты связаны смыслом, зависимостями и порядком объяснения. Один узел может вводить идею, другой — уточнять ограничения, третий — давать реализацию, и читатель движется по этой сети осмысленно, а не по принуждению компиляторного порядка.

Ретроспектива: Этот раздел выглядит методологически дальновидным. Это не прямое предсказание современных инструментов, но модель Дональда Кнута точно ложится на позднейшие задачи навигации в сложном коде: графы зависимостей, карты модулей, переходы по символам в IDE, call/reference-навигацию, knowledge graphs и контекстные карты проекта; LLM-контекст-карты здесь можно упомянуть только как предварительный горизонт. Для современной практики это означает простую вещь: понимание программы всё чаще строится через сеть связей между фрагментами, а не через линейный просмотр файлов.

WEB-модель не сводится к красивой метафоре «всё связано со всем». Её ценность в том, что сеть отношений фиксируется в проверяемом источнике, из которого получают и машинный результат, и человеческое объяснение. Именно поэтому раздел J в нашей логике стоит между переносимостью и стилем: сначала мы удерживаем целостность источника между платформами, затем понимаем его как сеть, и только после этого обсуждаем, как эту сеть называть и оформлять.

¹⁰ Build preset — именованный профиль сборки с заранее заданными параметрами, например компилятором, режимом и каталогом вывода.

K. Stylistic Issues

В разделе K Дональд Кнут переходит от архитектуры WEB к стилю работы внутри неё. Если программа становится объяснительным текстом, то стиль перестаёт быть косметикой — он становится частью инженерии понимания.

Главный вопрос раздела — как называть и организовывать секции. Дональд Кнут различает макросы и именованные фрагменты: маленькие технические сокращения можно оставить макросами, но более крупные части программы должны получать человеческое имя. Такое имя не обязано быть формально полным; оно должно достаточно точно выражать смысл фрагмента, не перегружая читателя деталями.

Плохое имя секции в poweb-like C++-ремастере:

```
<<code-for-loop>>
```

Лучшее имя:

```
<<generate prime candidates>>
```

Ещё лучше, если фрагмент действительно выражает действие:

```
<<print one page of the prime table>>
```

Дональд Кнут подчёркивает баланс между формальным и неформальным изложением. Нельзя превращать имя секции в полную спецификацию всех переменных и предпосылок: тогда оно перестаёт помогать и начинает имитировать юридический договор. Но нельзя и оставлять его слишком общим. Хорошая секция должна быть достаточно точной, чтобы читатель понял её роль до чтения кода.

Для C++-версии это значит, что фрагменты должны называться не по синтаксису, а по назначению:

```
<<constants>>  
<<types>>  
<<declarations>>
```

— допустимы для служебных блоков, но для логики лучше использовать смысловые имена:

```
<<generate the requested number of primes>>  
<<test whether a candidate is prime>>  
<<print the prime table page by page>>
```

Отдельно Дональд Кнут обсуждает управляющие структуры. Если поток управления нестандартен, имя секции должно это явно показывать. Опасен не сам `goto`, цикл или ранний выход, а неописанный смысл управляющего перехода. В современной C++-версии это касается `break`, `continue`, `return`, исключений, `guard clauses` и других способов менять обычное течение функции.

Во всех ветвях стиль остаётся рабочим инструментом: чем легче навигация по именам и ролям фрагментов, тем устойчивее понимание программы.

Позднейшая инженерная культура развивает похожую дисциплину в `naming conventions`: именах классов, методов, API, модулей, UI-блоков и БЭМ-подобных схемах¹¹, закреплённых `style guides` и практиками `readable code`. Это не `literate programming` само по себе, но та же борьба за читаемую структуру: имя должно сообщать роль фрагмента до погружения в реализацию.

Таким образом, этот раздел проводит границу между `literate programming` и современными документационными генераторами. `Javadoc`, `Doxygen` или извлечение комментариев могут создать справочник API, но это не обязательно `literate programming`. Они чаще описывают уже существующую структуру кода. У Дональда Кнута стиль работает на опережение.

L. Economic Issues

В разделе L Дональд Кнут задаёт прагматичный вопрос: сколько стоит WEB. Сначала он считает прямые вычислительные расходы: TANGLE занимает время, сопоставимое с компиляцией полученного Pascal-файла, WEAVE тоже работает достаточно быстро, но TeX-типографика уже заметно дороже. Поэтому полностью пересобирать и печатать документацию по несколько раз в день не всегда разумно.

Документацию сегодня печатают редко, но главный экономический аргумент находится не на уровне процессорного времени или затрат на бумагу. Дональд Кнут утверждает, что общее время написания и отладки WEB-программы у него не больше, чем время написания и отладки обычной ALGOL- или Pascal-программы, хотя документации получается значительно больше. Дополнительное время на

¹¹ БЭМ, или Block-Element-Modifier, — соглашение именования UI-компонентов, где имя показывает блок, его элемент и вариант состояния или модификации.

объяснение возвращается через сокращение отладки в связи с тем, что режим изложения заставляет прояснять мысли раньше. Когда программа пишется «для себя», легко использовать сокращения, которые позже становятся ошибками. Когда программа пишется как объяснение, автору труднее обмануть самого себя.

Для poweb-like C++-ремастера это означает, что цена метода должна быть честно названа:

```

;
tangle/render workflow;
canonical source;
generated .cpp      ;
.

```

Но выгода тоже конкретна:

```

;
;
;
,
.

```

Экономика в ветвях различается по порогу входа и типу рисков, но общий обмен работает по той же модели: больше дисциплины на входе, меньше стоимости сопровождения позже.

В этом разделе находится мост к современной reproducibility-практике. В research/data science literate-подход стал экономически оправданным, потому что отчёт, код, графики, результаты и проверка начали собираться из одного источника. В массовой практике это видно по исполняемым документам и воспроизводимым вычислениям: идея стала повседневной для науки и data science.

Финальная ирония раздела — опасение Кнута, что literate programs могут казаться авторам настолько завершёнными, что они будут стремиться публиковать их повсеместно: когда код становится текстом, появляется соблазн эстетизации.

Для обзора важно удерживать границу: ценность не в том, что программа выглядит как «произведение искусства», а в том, что она становится лучше объяснена и лучше проверяема.

M. Related Work

В разделе М Дональд Кнут снимает с WEB ореол одиночного открытия. Он прямо пишет, что в системе нет ничего «по-настоящему нового»: он собрал идеи, которые уже давно существовали, и соединил их в рабочую форму. Это важная авторская честность: WEB представлен как узел в истории программирования, типографики и публикации алгоритмов.

Первый важный источник — Джордж Форсайт, для которого полезный алгоритм является вкладом в знание, а его публикация — научной работой. Это прямо поддерживает мысль Дональда Кнута: программа может быть не только служебным механизмом, но и публикуемым интеллектуальным объектом. Отсюда естественно вырастает идея программы как литературы, но без романтизации: речь о научной форме представления алгоритма.

Далее Дональд Кнут называет Пьера-Арнуля де Марнеффа и holon programming, затем Дейкстру, Хоара, Даля, Вирта и Наура. Здесь видна основная интеллектуальная линия: абстракция, структурное программирование, баланс формального и неформального описания, публикация хорошо написанных программ. WEB не отменяет эту традицию, а пытается дать ей инструментальную форму.

Особенно важен эпизод с Тони Хоаром, который предложил Дональду Кнуту опубликовать TeX и для Кнута это был вызов: одно дело — опубликовать отполированные игрушечные задачи, другое — открыть реальную крупную программу со всеми компромиссами, переносимостью и грязными деталями. Именно эта проблема и рождает практическую потребность в WEB: сделать большую программу доступной для публичного чтения.

Этот раздел показывает, что literate programming возникло на пересечении нескольких линий:



Современная картина родственных практик устроена похожим образом. CWEB, noweb, Org Babel, R Markdown, Quarto и Jupyter не являются одной прямой линией.

Это скорее семейство практик, где разные области по-своему решают родственные инженерные задачи. Эту картину удобно читать как двухслойное влияние: прямые или исторически близкие продолжения WEB — прежде всего CWEB и noweb — и более массовый слой executable documents и reproducible computing, решающий родственные задачи иными средствами.

Поэтому в финале важно не искать одного «настоящего наследника». CWEB ближе по генеалогии, noweb — по простоте механизма, Org Babel — по plain-text-практике, Quarto/Jupyter/R Markdown — по институциональному масштабу. Каждый по-своему реализует часть той же инженерной задачи. Существенно лишь не смешивать прямое продолжение WEB, инструментальное родство и продуктивную смысловую ассоциацию.

В конечном счёте, WEB — это синтез, а не одиночное изобретение, что в свою очередь узаконивает сравнительный подход: современные ветви аналогично нужно читать не как копии WEB, а как разные способы продолжить центральный вопрос — как сделать программу читаемой и проверяемой.

N. Retrospect and Prospects

В финальном разделе Дональд Кнут сам снижает градус собственного манифеста. Он признаёт, что авторы новых языков обычно склонны переоценивать свой опыт, а его собственная работа с WEB слишком окрашена личными вкусами.

Статья заканчивается не победной декларацией, а осторожной проверкой — сработает ли метод за пределами автора и его среды.

При этом Дональд Кнут формулирует сильный итог: WEB, по его опыту, позволяет создавать программы более переносимые, понятные и удобные для сопровождения, причём метод работает не только на малых примерах, но и на больших системах. Для него WEB — результат обратного хода типографики: сначала компьютер применялся к набору текста, затем типографика вернулась в центр computer science как способ сделать программы читаемыми.

Дальше Дональд Кнут честно ограничивает аудиторию. WEB не проектировался для всех. Пользователь должен быть готов одновременно иметь дело с несколькими языками: WEB, TeX, Pascal и собственно алгоритмическими ошибками. Это не low-friction-инструмент, а среда для людей, которым нравится писать и объяснять, что они делают. В этом месте хорошо видно, почему классический WEB не стал массовым стандартом обычной разработки.

Но прогноз Дональда Кнута оказался точнее, чем может показаться. Он говорит о будущем, где философия WEB будет встроена в более эффективные программные среды: *tangling* и *compiling* могут объединиться, отладка может идти в терминах WEB-источника, секции программы можно будет показывать по требованию, а не печатать весь документ целиком. Это уже напоминает IDE, *symbol navigation*, *selective rendering*, *notebook*-среды и современные контекстные инструменты.

Эти линии затем собираются в двухконтурный итог: *WEB-like source-generation* и *executable-document narrative*.

Так финальный раздел переводит частный опыт WEB в более широкий вывод: классический WEB остался нишевым, но его принципы распространились шире.

Вывод. 1. Что именно предложил Дональд Кнут

Главное предложение Дональда Кнута не сводится к улучшению комментариев. Комментарий обычно остаётся вторичным слоем: он поясняет код, который уже организован в порядке, удобном компилятору, языку или привычке автора. *Literate programming* предлагает более сильную модель: программа должна изначально строиться как объяснительный документ, рассчитанный на человеческое чтение.

В этой модели центральным объектом становится не сгенерированный *.pas*, *.c*, *.cpp* или *.html*, а единый источник, из которого выводятся разные представления:

```
literate source
  → machine-oriented artifact
  → human-oriented document
```

У Дональда Кнута эта схема реализована через WEB, TeX, Pascal, TANGLE и WEAVE, но принцип шире исходной технологической пары. Важна не конкретная историческая связка, а дисциплина: объяснение, код и механизм получения артефактов должны оставаться связанными. Именно поэтому программа у Дональда Кнута перестаёт быть линейным файлом. Она становится сетью именованных фрагментов, где порядок изложения может следовать не машинной последовательности, а логике понимания.

Код здесь является частью объяснительной структуры; это и остаётся главным критерием для современных продолжений и переосмыслений *literate programming*.

Вывод. 2. Что сохранилось, а что сменило форму

Классический WEB не стал массовым стандартом разработки, но несколько его принципов оказались устойчивыми. Они продолжили существовать не в одной линии, а в разных инженерных и исследовательских практиках.

Первая линия — **source-generation**. CWEB, noweb и Org Babel сохраняют наиболее близкую к WEB идею: существует структурированный источник, из которого можно получить машинно-ориентированный код. В этой линии особенно важны именованные фрагменты, tangling и запрет на ручное редактирование производного артефакта.

Вторая линия — **executable documents**. Quarto, Jupyter и R Markdown смещают акцент с генерации программного исходника на воспроизводимый документ, где текст, код, выполнение и результаты находятся в одном контуре. Это уже не буквальный WEB, но задача остаётся родственной: вычисление должно быть не только выполнено, но и объяснено, пересобрано и проверено.

Третья линия — **навигация по сложным системам**. Раздел Дональда Кнута о программах как сетях оказался особенно дальновидным. Современные IDE, dependency graphs, symbol navigation, module maps, knowledge graphs и context maps работают с похожей проблемой: программа понимается не через линейное чтение файлов, а через сеть связей между фрагментами.

Четвёртая линия — **воспроизводимое окружение**. В классическом WEB переносимость поддерживалась через master source и change files. В современной практике ту же роль частично выполняют WSL, контейнеры, CI, dev containers, lock-files, build presets и документированные команды запуска. Companion-набор этого обзора был проверен в WSL-среде под Windows 11, что хорошо показывает новую форму старого вопроса: воспроизводить нужно не только код, но и окружение, в котором код становится проверяемым артефактом.

Поэтому точнее говорить не о едином наследнике WEB, а о распределённой картине продолжений, переосмыслений и смысловых ассоциаций. 01-cweb, 02-noweb-like и 03-org-babel образуют WEB-like / source-generation линию. 04-quarto, 05-jupyter и 06-rmarkdown образуют executable-document / computational-publishing линию. Эти ветви различаются технически, но каждая по-своему продолжает центральный вопрос Дональда Кнута: как связать программу, объяснение и проверяемый результат.

Вывод. 3. Цена метода и границы применимости

Literate programming не является универсальной заменой обычной разработки. Его сила начинается там, где понимание программы становится частью производственного результата: в сложных алгоритмах, исследовательских вычислениях, системах с долгим жизненным циклом, предметных правилах, генераторах, DSL, архитектурных пайплайнах, обучающих материалах и проектах, где цена ошибки выше стоимости предварительного объяснения.

Ограничения также существенны. Классический WEB требует владения несколькими языками и режимами работы: языком документа, языком программирования, синтаксисом literate-системы и самой предметной задачей. Даже облегчённый poweb-like подход сохраняет цену дисциплины: нужно поддерживать canonical source, выполнять tangling/rendering, не редактировать generated artifacts вручную и регулярно проверять результат.

Эта цена не является дефектом метода; она определяет его область применимости. Для коротких скриптов, одноразовых утилит и тривиального CRUD-кода полный literate-процесс часто будет избыточен. Для систем, которые нужно объяснять, сопровождать, проверять и передавать другим людям, он становится инженерным способом уменьшить будущую стоимость непонимания.

Здесь уместна осторожная связь с языком инженерных паттернов. Literate programming не является GoF-паттерном в строгом каталоговом смысле (Gamma и др. 1994). Но его можно рассматривать как устойчивый паттерн организации source и мышления: повторяющееся решение проблемы, где код, объяснение и проверка должны оставаться связанными. В этом смысле он ближе не к конкретному объектно-ориентированному шаблону, а к методологической рамке проектирования.

На этом заканчивается собственно ретроспективная часть вывода. Далее обзор сознательно расширяет рамку: речь пойдёт уже не только о том, какие формы literate programming существовали исторически и какие линии можно проследить в современных инструментах, но и о том, как центральный принцип Дональда Кнута может быть переосмыслен в условиях LLM-assisted разработки. Это не утверждение о прямом историческом наследовании, а авторское развитие поставленной Кнута проблемы.

Вывод. 4. LLM: продолжение идеи или новая форма разрыва

Большие языковые модели (LLM) делают вопрос Дональда Кнута заново острым. Если источником истины становится только prompt, literate programming не возникает. Получается быстрая генерация кода по намерению, но не обязательно объяснимый, проверяемый и сопровождаемый source. Prompt может быть полезным входом, но сам по себе он редко фиксирует архитектуру, ограничения, инварианты, тестовые сценарии и трассировку принятого решения.

Разница между TANGLE и LLM принципиальна. TANGLE детерминированно преобразует структурированный source в программный артефакт. LLM вероятно продолжает текст на основании контекста, инструкций и примеров. Поэтому LLM-вывод не может быть равен ветке сборки: он должен рассматриваться как candidate artifact, который проходит review, тесты и включается обратно в управляемый source.

Сильная современная версия выглядит так:

```
human-authored literate plan
  → chunk contracts
  → bounded prompt
  → LLM-generated candidate
  → review
  → tests / smoke-check
  → TRACE
  → accepted update to canonical source
```

В этой формуле модель не является архитектором по умолчанию. Она работает внутри структуры, заданной человеком: секции, чанки, ограничения, ожидаемое поведение и критерии принятия должны существовать до генерации или уточняться до повторного запуска. Иначе LLM не продолжает literate programming, а лишь ускоряет производство кода, который потом приходится заново понимать.

Соседние практики — scripting, dialogue systems, rule engines, query parsers, предметно-ориентированные языки (DSL)¹² и behavior trees — помогают увидеть общий инженерный мотив. Между человеком и машиной часто появляется промежуточный язык намерений. Но в DSL или rule engine этот язык обычно формализован, тогда как LLM добавляет вероятностную интерпретацию. Поэтому

¹² Domain-specific language, или DSL, — язык, специально ограниченный задачами конкретной предметной области.

для AI-assisted разработки дисциплина source, contracts, tests и traceability становится более важной.

Именно здесь проходит граница с vibe coding. Простая схема prompt → code может быть продуктивной для черновика, но сама по себе не продолжает literate programming. Содержательным продолжением может быть только workflow, где prompt подчинён объяснительному source, а результат проходит проверку и возвращается в трассируемую структуру проекта.

Поэтому в companion-наборе нет детерминированного 07-11m, равного CWEB, noweb, Org Babel, Quarto, Jupyter или R Markdown. Вместо него введён 07-prompt-literate как демонстрационное применение **Prompt-Literate Workflow**. Его задача не в том, чтобы воспроизвести LLM-вывод бит-в-бит, а в том, чтобы показать управляемый контур: human-authored plan, chunk contracts, bounded prompts, review, smoke-check и TRACE.

07-prompt-literate не определяет Prompt-Literate Workflow заново. Он является локальным примером применения метода в рамках статьи к задаче Дональда Кнута о простых числах. Специфические маркеры результата и локальные проверки оформлены как additive-расширение поверх базовой методологии: оно добавляет локальные проверки, но не переопределяет базовые инварианты.

Индустриальные инструменты уже движутся в эту сторону: VS Code / GitHub Copilot custom instructions, prompt files, agent customization, skills/hooks и MCP-подобные механизмы¹³ формируют устойчивый проектный контекст («Customize GitHub Copilot in Visual Studio Code», б. д.; «Customizing GitHub Copilot», б. д.). Отдельный контекст создают регуляторные требования и provenance-стандарты¹⁴: EU AI Act¹⁵ и General-Purpose AI Code of Practice¹⁶ усиливают запрос на прозрачность и ответственность, а C2PA¹⁷ задаёт техническую модель фиксации происхождения и истории изменений цифрового контента. Но это пока признаки переходной стандартизации, а не готовое доказательство того, что AI-coding уже стал literate programming.

¹³ Model Context Protocol, или MCP, — открытый протокол подключения AI-приложений к внешним системам, источникам данных и инструментам.

¹⁴ Provenance — сведения о происхождении, авторстве и цепочке изменений артефакта.

¹⁵ EU AI Act — регламент Европейского союза, задающий риск-ориентированные требования к системам искусственного интеллекта.

¹⁶ General-Purpose AI Code of Practice — добровольный рамочный инструмент ЕС, помогающий провайдерам моделей общего назначения выполнять требования EU AI Act.

¹⁷ C2PA — техническая спецификация Coalition for Content Provenance and Authenticity для машинно-читаемой фиксации происхождения и истории изменений цифрового контента.

Вывод. 5. Где идея может работать дальше

Наиболее очевидная область применения *literate*-подхода сегодня — научные и аналитические документы. Там связка текста, кода и вычисления, родственная постановке Дональда Кнута, уже стала повседневной практикой: R Markdown, Quarto, knitr, Org Babel и *notebook*-среды связывают текст, код, данные, вычисления, графики и публикацию. В этих областях объяснение не является украшением; оно необходимо для проверки результата.

Вторая область — обучение. *Literate*-подход особенно полезен там, где нужно показать не только итоговый код, но и ход мысли: алгоритмы, структуры данных, системное программирование, C++, разбор *legacy*-кода. Программа становится не «правильным ответом» в конце учебного материала, а маршрутом понимания.

Третья область — архитектурная и инструментальная документация: *build pipelines*, *code generation*, DSL, конфигурационные системы, *runtime contracts*, тестовые сценарии и воспроизводимые окружения. Там обычная документация рядом с кодом быстро устаревает, если не связана с артефактами и проверками. *Literate source* не решает эту проблему автоматически, но заставляет явно определить источник истины.

Четвёртая область — *game development* и пайплайны *game design document* (GDD)¹⁸. Здесь идея Дональда Кнута может быть расширена за пределы собственно программирования. GDD уже находится между литературным описанием, спецификацией, системой правил и будущей реализацией. Хороший GDD не просто рассказывает об игре: он должен порождать сцены, механики, данные, квестовые структуры, тестовые сценарии, задачи и проверяемые артефакты.

В этой точке появляется более широкая рамка — ***literate design***:

```
GDD source
→ narrative documentation
→ mechanics/spec chunks
→ quest structures
→ data schemas
→ implementation tasks
→ validation checks
→ generated wiki / build artifacts
```

¹⁸ Game design document, или GDD, — проектный документ игры, связывающий замысел, правила, контент, системы и будущую реализацию.

Это не буквальное наследование WEB, но оно продолжает ранний вопрос в иной области: как описать сложную систему в форме, из которой можно получать рабочие представления и проверяемые артефакты.

Для наших проектов этот обзор имеет прикладной смысл. Он возник не как ностальгический комментарий к статье 1984 года, а как подготовка к пайплайну, где единый объяснительный документ может связывать GDD, код, тесты, wiki, generated artifacts и LLM-assisted работу. У нас source должен быть достаточно выразительным, чтобы объяснять систему, и достаточно дисциплинированным, чтобы порождать проверяемые результаты.

Практическое применение показало, что универсальный workflow должен оставаться компактным. Доменные правила — архитектура конкретного проекта, runtime-интеграция, persistence, authoring tools и publication pipeline — подключаются как локальные для проекта расширения и ни в коем случае не должны становиться частью базового метода.

Literate programming действительно нужен не везде. Но там, где понимание является частью производства, его центральная идея остаётся актуальной: сначала построить объяснительный источник, затем получить из него машинные и человеческие представления, а не пытаться реконструировать смысл после того, как код уже возник.

А. Первичные источники: Кнут, WEB/CWEB, структурное программирование и контекст проектирования ПО

Dijkstra, E.W. 1972: 'Notes on Structured Programming'. In O.-J. Dahl et al. (eds), *Structured Programming*. Academic Press, 1–82.

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. 1994: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Knuth, D.E. 1984: 'Literate Programming'. *The Computer Journal*. DOI: [10.1093/comjnl/27.2.97](https://doi.org/10.1093/comjnl/27.2.97).

Knuth, D.E. and Levy, S. n.d. 'The CWEB System of Structured Documentation'. <https://www-cs-faculty.stanford.edu/~knuth/cweb.html>.

Martin, R.C. 2017: *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, 1st edn. Prentice Hall.

В. Прямые инструменты literate programming и их наследники

‘GNU Emacs Download’ n.d. <https://www.gnu.org/software/emacs/download.html>.

‘Org Manual: Extracting Source Code’ n.d. <https://orgmode.org/manual/Extracting-Source-Code.html>.

Ramsey, N. n.d. ‘Noweb Repository’. <https://github.com/nrnrrnr/noweb>.

С. Исполняемые документы, воспроизводимые исследования и notebook-линия

‘Knitr’ n.d. <https://yihui.org/knitr/>.

‘Nbconvert Documentation’ n.d. <https://nbconvert.readthedocs.io/>.

‘Pandoc’ n.d. <https://pandoc.org/>.

‘Project Jupyter Documentation’ n.d. <https://docs.jupyter.org/>.

‘Quarto Documentation’ n.d. <https://quarto.org/docs/>.

‘R Markdown Documentation’ n.d. <https://rmarkdown.rstudio.com/>.

D. LLM и AI-assisted programming

‘Customize GitHub Copilot in Visual Studio Code’ n.d. <https://code.visualstudio.com/docs/copilot/copilot-customization>.

‘Customizing GitHub Copilot’ n.d. <https://docs.github.com/en/copilot/how-tos/custom-instructions>.

Е. Управление и provenance-контекст

‘C2PA Specification’ n.d. <https://c2pa.org/specifications/>.

‘General-Purpose AI Code of Practice’ n.d. <https://digital-strategy.ec.europa.eu/en/policies/ai-code-practice>.

Regulation (EU) 2024/1689 (Artificial Intelligence Act) n.d. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.